

BLACKOUT Protocol

Team Task Separation & Development Setup

Purpose

This document divides the project between six members using the two-members-per-level strategy. Each pair owns one complete level, while selected members also own shared systems that the whole game depends on. The goal is to allow everyone to work in parallel while still producing one game that can be merged and that covers the project rubric.

The Three-Pair Structure

Pair	Level	Member	Main responsibility
Pair 1	Level 1 — Subway	1A	Core systems, Kai/player, camera
		1B	Subway world, lighting, shader, audio
Pair 2	Level 2 — Highway	2A	Car, Handler AI, driving gameplay
		2B	Highway world, reflections, cameras, shader
Pair 3	Level 3 — Mine	3A	Combat, Handler boss, transitions
		3B	Mine world, UI, story/ending, shaders

1. Member 1A — Core Systems + Kai / Level 1

Main idea: build the systems that the other levels inherit rather than rewriting them.

- Create the Three.js project structure and main game loop.
- Create shared game state: menu → Level 1 → Level 2 → Level 3 → game over → complete.
- Build Kai's player controller: movement, lane snapping, jump, slide and collision.
- Implement keyboard/mouse input and the shared Input system.
- Build the main chase camera and camera hierarchy following Kai.
- Create level loading and transitions without refreshing the browser.
- Implement restart without reloading the page.
- Use Object3D hierarchy for player/camera relationships.

Recommended tools

- Three.js
- Vite + JavaScript
- Git/GitHub
- Rapier or cannon-es
- AnimationMixer

Rubric coverage

Viewing/cameras, Control & Playability, and Polish. This member also ensures shared systems are compatible with Levels 2 and 3.

2. Member 1B — Level 1 Subway + Lighting + Audio

- Build subway/tunnel, tracks, platforms, pipes, walls, obstacles and security gate.
- Build the service area and place the transit maintenance vehicle that starts Level 2.
- Use multiple lights, shadows, fog and emissive elements.
- Use textured materials with normal/roughness maps rather than colour-only materials.
- Create Shader 1: speed/warp effect driven by player speed.
- Implement audio: subway ambience, footsteps, Handler sounds, engine/cave sounds and music transitions.

Recommended tools

- Blender → .glb/.gltf
- Three.js MeshStandardMaterial
- DirectionalLight / SpotLight / HemisphereLight
- EffectComposer / ShaderPass
- Three.js PositionalAudio

Rubric coverage

3D Effects, Shaders, and Gameplay & Experience through atmosphere, sound, lighting and visual consistency.

3. Member 2A — Level 2 Vehicle + Handler AI

- Build vehicle acceleration, steering, braking, collision and drifting.
- Add boost and a heat/energy system.
- Create traffic and hazard spawning while reusing the shared chunk system.
- Build Handler AI states: follow/close, ram and additional attacks.
- Give attacks visible telegraphs so the player can understand and dodge them.
- Implement damage and health systems.
- Create the mountain-bridge ram that sends Kai into the ravine and ends Level 2.
- Tune difficulty so the game becomes challenging without becoming unfair.

Recommended tools

- Rapier or cannon-es
- Three.js
- Finite-state machine pattern
- Raycasts/collision detection
- Shared GameState and Input

Rubric coverage

Control & Playability, Gameplay & Experience, and Innovation through the transition from running to driving and through varied pursuer attacks.

4. Member 2B — Level 2 Highway + Visual Pipeline

- Build highway chunks, guardrails, traffic scenery, streetlights, city skyline and mountain road.
- Set shared art pipeline: Blender scale, naming, export settings, texture limits and asset structure.
- Use InstancedMesh for repeated objects such as streetlights and barriers.
- Create dynamic dusk-to-night environment with matching lighting changes.
- Add reflections to Kai's car.
- Create Shader 2: rain-on-glass/windshield effect.
- Create rear-view picture-in-picture camera showing the Handler.
- Create the orthographic minimap.
- Protect frame rate by reducing unnecessary draw calls and oversized textures.

Recommended tools

- Blender
- Three.js InstancedMesh
- CubeCamera/reflection tools
- EffectComposer
- RenderTarget
- OrthographicCamera

Rubric coverage

Viewing, 3D Effects, Shaders and Polish/performance.

5. Member 3A — Level 3 Combat + Handler Boss + Transitions

- Build free-arena movement using the shared input system.
- Implement attack, block, parry, dodge, lock-on and stamina.
- Implement hit detection, hit-stop and camera shake.
- Create Handler boss with three phases: Pursuit, Stand and Desperation.
- Trigger the helmet-off reveal during the boss fight.
- Build Level 1 → Level 2 service-vehicle transition.
- Build Level 2 → Level 3 bridge-crash transition.
- Use scripted camera paths rather than pre-rendered videos where practical.

Recommended tools

- Three.js
- AnimationMixer
- Raycaster/sphere casts/trigger volumes
- CatmullRomCurve3
- Shared Input and GameState

Rubric coverage

Control & Playability, Gameplay & Experience, and Innovation. The genre change from runner → driving → combat should be a major demonstration point.

6. Member 3B — Level 3 Mine + UI + Story + Release

- Build ravine, mine entrance, underground tunnels, rocks, stalagmites, water/lava areas and hidden server facility.
- Use normal and height maps and instancing for repeated cave geometry.
- Create Shader 3: lava/heat distortion.
- Create Shader 4: dissolve/digital effect.
- Build main menu, pause menu, options, loading screen and HUD.
- Implement story cards and level-complete screens.
- Implement ending: BLACKOUT PROTOCOL: DEACTIVATED → twelve authorized keys → 11 KEYS REMAIN.
- Own build/release process, credits screen and asset log.
- Capture/edit trailer footage with footage supplied by the whole team.

Recommended tools

- Blender
- Three.js materials/shaders
- HTML/CSS/JS UI
- LoadingManager
- Vite

- Git/GitHub

Rubric coverage

Polish, Shaders, Gameplay & Experience, story delivery, trailer support and credits/asset-log requirement.

7. How the Six Members Work in Parallel

The two-members-per-level split only works if everyone agrees on the shared interfaces before building their level. The team should have one shared repository and one shared project rather than six separate projects.

Every level should follow the same basic contract:

```
class Level {  
    init(scene, assets, input) {}  
    update(dt, gameState) {}  
    teardown() {}  
}
```

Shared modules

- Input: one shared keyboard/mouse/game-control system. All levels call the same input functions.
- GameState: one shared object containing current level, health, distance, boost/stamina, collected letters and game phase.
- AssetRegistry: one agreed way to load and access models, textures, sounds and other assets.
- Level interface: every level exposes init(), update() and teardown().
- Common naming and folder conventions: everyone follows the same asset and code naming rules.

Suggested project structure

```
BLACKOUT/  
■■■ src/core/ – Game, GameState, Input, AssetRegistry, Level  
■■■ src/player/ – Kai/player systems  
■■■ src/levels/ – Level01, Level02, Level03  
■■■ src/ui/ – menus, HUD, story cards  
■■■ src/audio/ – shared audio system  
■■■ src/shaders/ – shared/custom shaders  
■■■ assets/ – level01, level02, level03, characters, textures
```

Git workflow

- Everyone clones the same GitHub repository.
- Each member works on their own branch.
- Do not directly overwrite another member's files.
- Pull/rebase from main regularly.
- Merge only after testing the branch.
- Keep commits small and descriptive.

8. Software and Tools Everyone Should Install

- VS Code — main code editor.
- Node.js — required to run the Vite/Three.js project.
- Git — version control.
- GitHub access — shared repository and branches.
- Blender — 3D modelling and asset preparation.
- A current Chrome/Edge browser for testing.
- The same project dependencies and Node.js version for everyone.
- One shared physics library: Rapier OR cannon-es. Do not mix physics engines between levels.

Asset rules everyone must follow

- Use .glb/.gltf for 3D models.
- Agree on one Blender unit/scale convention.
- Use consistent lowercase asset names.
- Keep textures within the team's agreed size limit.
- Do not commit unnecessary duplicate or huge assets.
- Keep assets in the correct level/character/texture folders.
- Record externally sourced assets in the asset log for final credits.

9. Rubric Ownership Map

Rubric area	Primary owner	Supporting members
Viewing / cameras	1A	2B, 3A
Control & Playability	1A	2A, 3A
3D Effects	1B	2B, 3B
Shaders	1B	2B, 3B
Gameplay & Experience	2A	1B, 3A, 3B
Innovation	3A	All members
Polish / performance	2B + 3B	All members
Story	3A + 3B	All members
Audio	1B	All members
Trailer	3B	All members provide footage
Credits / asset log	3B	Everyone supplies asset information

Important: Ownership means one member is accountable for making sure the rubric area is completed. It does not mean the member is the only person allowed to contribute to it.

10. Day-One Team Checklist

- Create the shared GitHub repository.
- All six members clone the same project.
- Install the same Node.js version and dependencies.
- Create and agree on the folder structure.
- Agree on the Level interface: `init()`, `update()`, `teardown()`.
- Agree on Input functions and keyboard mappings.
- Agree on GameState fields and naming.
- Agree on Blender scale and .glb export settings.
- Agree on asset naming and texture limits.
- Choose one physics engine.
- Create a tiny test scene that loads one shared asset and proves the project runs on every member's computer.
- Each pair creates a sandbox version of its level without changing the shared interface.
- Merge frequently rather than waiting until the end.

The goal

By following this setup, the three pairs can genuinely work in parallel: Pair 1 can build the subway and core systems, Pair 2 can build the highway and vehicle, and Pair 3 can build the mine and combat. Because all three levels use the same Input, GameState, AssetRegistry and Level interface, they can later be connected into one continuous BLACKOUT game instead of becoming three separate prototypes.